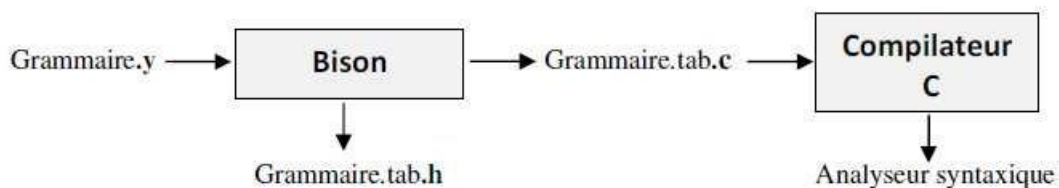


TP2 - Analyse syntaxique

Partie I – Analyseur Syntaxique : Bison

Le rôle de l'analyseur syntaxique est de vérifier la syntaxe du programme source. Il reçoit une suite d'unités lexicales fournie par l'analyseur lexical et doit vérifier que cette suite peut être engendrée par la grammaire du langage.

Bison (version GNU de YACC) est un générateur d'analyseurs syntaxiques. Il accepte en entrée la description d'un langage sous forme d'une grammaire et produit un programme écrit en C qui, une fois compilé, reconnaît les mots (programmes) appartenant au langage engendré par la grammaire en entrée.



Un fichier de spécifications Bison se compose de quatre parties :

```
%{  
  Les déclarations en c  
}%  
Déclaration des Unités lexicales utilisées  
Déclaration de priorités et de types  
%%  
Règles de production avec éventuellement des actions sémantiques  
%%  
Bloc principal et fonctions auxiliaires en C
```

Règles de production :

```
Non-terminal : prod1  
              | prod2  
              | prod3  
              ....  
              | prodn  
              ;
```

Les symboles terminaux sont :

- Des unités lexicales (qu'on doit impérativement déclarer dans la partie 2)
Syntaxe : `% token nom-unité-lexicale`
Exemple : `% token NB`
 `% token ID`
- Des caractères entre quotes : '+', 'a'...
- Des chaînes de caractères entre guillemets : "while"

Les symboles non-terminaux représentent toute suite de lettres majuscules et/ou minuscules.

Remarque: La partie *Bloc principal et fonctions auxiliaires* doit contenir une fonction **yylex()** effectuant l'analyse lexicale du programme source. On peut soit écrire directement cette fonction soit utiliser la fonction produite par Flex.

Partie II – Installation de l’environnement et Tests

a. Télécharger Bison :

- Rendez-vous sur le site de Cygwin <https://www.cygwin.com/>
- Téléchargez le fichier suivant : [setup-x86_64.exe](#)
- **Installer le fichier téléchargé** Pendant l'installation, vous devrez sélectionner les packages nécessaires (recherchez **Bison** dans la liste des packages et assurez-vous de sélectionner le package **bison**)

b. Configurer le chemin (PATH) :

- Ajoutez le chemin du dossier bin de GnuWin32 à la variable d'environnement **PATH** :
 - i. Ouvrez les **Paramètres système avancés**.
 - ii. Cliquez sur **Variables d'environnement**.
 - iii. Modifiez la variable **PATH** et ajoutez le chemin : C:\cygwin64\bin

c. Vérifier l'installation :

- Ouvrez l'invite de commandes (**cmd**) et tapez :
`bison --version`

1. Ouvrir un nouveau fichier texte et taper le code ci-dessous. Le fichier doit être enregistré avec l'extension **.y** (par exemple *grammaire.y*).

a. A partir de l'invite de commande, lancer la commande :

bison grammaire.y

b. En cas de réussite, le fichier *grammaire.tab.c* est généré dans le même répertoire.

c. Compiler le fichier *grammaire.tab.c* pour générer l'exécutable.

d. Quels sont les mots acceptés ?

```
%{
#include <stdio.h>
int yylex(void);
int yyerror(char*);
}%

%%
mot : S '$' { printf("mot correct\n"); getchar(); }
;

S : 'a' S 'a'
  | 'b' S 'b'
  | 'c'
;

%%

int yylex() {
    char c = getchar();
    if (c == 'a' || c == 'b' || c == 'c' || c == '$')
        return c;
    else {
        printf("Erreur lexicale\n");
        return 0;
    }
}
```

Coordination entre Flex et Bison

On peut utiliser la fonction *yylex()* générée par Flex pour effectuer l'analyse lexicale.

Remarque: Un attribut est associé à chaque symbole de la grammaire (terminal ou non). L'attribut d'une unité lexicale est la valeur contenue dans la variable globale prédéfinie **yyval**. Cette variable est l'outil de base de communication entre Flex et Bison au cours de l'analyse. Il faut donc penser à affecter correctement **yyval** lors de l'analyse lexicale. La définition de **yyval** est partagée dans le fichier ".h" issu de "bison -d".

Exemple:

```
%[0-9]+ {yyval=atoi(yytext); return NB ;}
%%
```

- Par défaut **yyval** est de type entier. On peut changer son type par la déclaration dans la partie 2 d'une union. Exemple :

```
% union {
int entier ;
double reel ;
char * chaine ;
}yyval ;
```

Dans ce cas, on peut stocker dans **yyval** des entiers, des réels ou bien des chaînes de caractères. Il faut typer les unités lexicales et les symboles non-terminaux dont on utilise l'attribut, soit l'exemple:

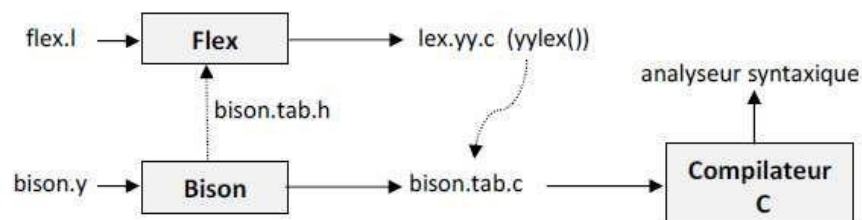
```
% token <entier> NB
% token <chaîne> ID
% type <entier> S
% type <chaîne> Expr
```

- Pour les symboles non-terminaux, \$\$ désigne la valeur de l'attribut associé au non-terminal de la partie gauche. \$i désigne la valeur associée à l'ième symbole non-terminal de la partie droite, par exemple :

```
Expr : Expr '+' Expr {temp=$1 + $3 ;} '*' Expr {$$=temp + $6 ;} ;
```

- Les attributs peuvent être utilisés dans les actions sémantiques.

La génération d'analyseur syntaxique à l'aide de Flex et Bison est illustrée par la figure ci-dessous :



Exercice 1

On veut écrire à l'aide de Flex et Bison un analyseur syntaxique qui permet de reconnaître les instructions de la forme suivante :

somme 1, 2, 3. produit 2,5. \$

Il s'agit d'une liste d'entiers séparés par des virgules, se terminant par un point et précédées soit par le mot somme soit par le mot produit.

1. Écrire tout d'abord le fichier de spécifications Bison suivant :

```
%{
#include<stdio.h>
int yylex(void);
int yyerror(char *s);
}%

%token FIN;
%token SOM;
%token PROD;
%token NB;

%%
liste:FIN {printf("correct");}
      |SOM listesom'.'liste
      |PROD listeprod'.'liste;
listesom: NB|listesom', 'NB;
listeprod: NB|listeprod', 'NB;
%%

#include "lex.yy.c"
int yyerror(char *s)
{
    printf ("%s", s);
    return (0);
}
int main()
{
    yyparse();
    getchar();
}
```

2. Compiler le fichier en utilisant l'option `-d`, par exemple `bison -d nom.y`
3. En cas de réussite, les fichiers *nom.tab.h* et *nom.tab.c* sont générés dans le même répertoire.
4. Inclure le fichier *nom.tab.h* dans le fichier de spécifications Flex (ce fichier contient la description d'unités lexicales utilisées).
5. Ouvrir un nouveau fichier texte et taper le code de spécifications Flex suivant :

```
%{
#include <stdio.h>
#include <stdlib.h> // Pour atoi()
#include "nom.tab.h" // Assurez-vous que ce fichier est généré par
Bison
}%

%%
[0-9]+      { yylval = atoi(yytext); return NB; }
produit     { return PROD; }
somme       { return SOM; }
[,]'        { return ','; }
[.]         { return '.'; }
[$]         { return FIN; }
[ \t\n]     { /* Ignore les espaces blancs */ }
.           { printf("Erreur lexicale : %s\n", yytext); }
%%

int yywrap() {
    return 1;
}
```

6. Compiler ce fichier avec Flex pour générer le fichier *lex.yy.c*.
7. Inclure le fichier *lex.yy.c* dans le fichier de spécifications Bison.
8. Compiler maintenant le fichier *nom.tab.c* pour obtenir l'exécutable.
9. Tester le fichier *nom.tab.exe* obtenu pour vérifier qu'il fonctionne correctement.
10. Modifier l'exercice précédent pour que l'exécutable affiche la somme (respectivement le produit) des entiers formant chaque suite.

Exemple : Si l'entrée est : *somme 2,5. Produit 3,6.\$*
Le résultat sera : *Somme = 7*
Produit=18

11. Modifier cet exemple pour construire un analyseur syntaxique permettant de faire la soustraction et la division.